

Naive Bayes types [Part-2]

What is underflow in Computing:-

underflow $\Rightarrow$ represent decimal in memory (very difficult)
 $\Downarrow$
binary (0 & 1)

if 0.00003 has to be stored in a memory means, it considers as zero.

$\rightarrow$ Underflow happens when a number becomes extremely close to zero, so small that the computer cannot represent it properly in floating-point format.

$\rightarrow$ Instead of storing the tiny value, the computer rounds it to 0, which causes loss of precision. Ex:- all radiation
0.00003 $<$ 0.0000018

Why does this happen:-

- $\rightarrow$ Computers use floating-point representation to store real numbers.
- $\rightarrow$ Floating-point numbers have a minimum positive limit.
- $\rightarrow$ If a computed value is smaller than that minimum, it becomes zero.
- $\rightarrow$ This is called numerical underflow.

Problem in Naive Bayes:-

2500 rows

Cgpa	iq	placements
-	-	- Y
		N
8.1	81	?

$$\Rightarrow \{8.1, 81\} \Rightarrow \{Y|N\}$$

$$P(Y|8.1, 81) \Rightarrow P(Y) P(8.1|Y) P(81|Y)$$

$\Rightarrow 0$ (2500 rows)

$$P(N|8.1, 81) \Rightarrow P(N) P(8.1|N) P(81|N)$$

$= 0$ Consider $\Rightarrow 0.1 \times 0.3 \times 0.6 \times 0.2 \times \dots$

$\rightarrow$ after computing the value may 0.00000000000001

$\rightarrow$ when we compare both probabilities, it will say both are equal.

$\rightarrow$ This problem is called as underflow.

Solution: we use logarithms

→ Instead of multiplying probabilities, we take log:

$$\log P(Y) + \log \sum \log P(x_i | Y)$$

• $\log(a \times b) \Rightarrow \log a + \log b \Rightarrow$ log converts multiplication to addition.

• Addition of negative numbers is numerically stable

• It prevents numbers from shrinking to zero

Ex:- $\log P(Y) + \log P(8.1 | Y) + \log P(81 | Y) + \dots$ 25000
 $\log P(N) + \log P(8.1 | N) + \log P(81 | N) + \dots$ 25000

$$\log(P(a) \times P(b) \times P(c) \dots) \Rightarrow \log P(a) + \log P(b) + \log P(c) + \dots$$

if multiplication:- $\log(0.3 \times 0.5 \times 0.7)$
 $\Rightarrow \log(0.3) + \log(0.5) + \log(0.7) \Rightarrow -1.2 + 0.7 - 0.3 = -2.2$

for - negative results $\Rightarrow \log P(Y) + \log(8.1 | Y) + \log P(81 | Y) \Rightarrow -1.53$
 $\log P(N) + \log P(8.1 | N) + \log P(81 | N) \Rightarrow -1.22$

highest prob is selected

General Laplace Additive Smoothing:-

In Naive Bayes (especially text classification), if a word never appears in a class during training, then:

$$P(\text{word} | Y) = 0$$

→ Since Naive Bayes multiplies probabilities, one zero makes the entire product zero → prediction collapses.

→ Laplace smoothing prevents this.

Ex:- binary bow

review	Sentiment		w_1	w_2	w_3	Sentiment	
$w_1 \quad w_2 \quad w_3$	0	$\Rightarrow$	01	1	1	0, -ve	
$w_1 \quad w_3 \quad w_3$	1		02	1	0	2	1, +ve
$w_2 \quad w_2 \quad w_1$	0		03	1	1	0	0, -ve

new data $\Rightarrow$ 014 $\rightarrow w_1, w_2, w_3$ $\rightarrow$ +ve, -ve

$$P(+ve | y_4) \Rightarrow P(+ve) P(w_1=1 | +ve) P(w_2=0 | +ve) P(w_3=0 | +ve)$$

$$\Rightarrow \frac{1}{3} \quad \frac{1}{1} \quad \frac{0}{1} = \boxed{0} //$$

$$P(-ve | y_4) \Rightarrow P(-ve) P(w_1=1 | -ve) P(w_2=0 | -ve) P(w_3=0 | -ve)$$

$$\frac{2}{3} \quad \frac{2}{2} \quad \frac{0}{2} \Rightarrow \boxed{0} //$$

$= \log(0) \Rightarrow \text{undefined}$

General additive smoothing:-

$$P(\text{word} | y) = \frac{\text{count} + \alpha}{\text{Total word} + \alpha V}$$

• $\alpha = 1 \rightarrow$ Laplace Smoothing

• $\alpha < 1 \rightarrow$ less smoothing

• $V \Rightarrow$ vocabulary smoothing

$\rightarrow$ This keeps probabilities properly normalized.

Ex:- Training data

Spam : "free money"

Ham : "meeting now"

Vocabulary = { free, money, meeting, now }

$$V = 4$$

$\rightarrow$ Suppose we test for word "free"

$\Rightarrow$ without smoothing :- If "free" never appear in Ham:

$$P(\text{free} | \text{Ham}) = 0 //$$

$$\Rightarrow \text{with smoothing} \Rightarrow P(\text{free} | \text{Ham}) = \frac{0+1}{2+4} \Rightarrow \frac{1}{6}$$

$\Rightarrow$ Now it is small, but not zero.

"Laplace Smoothing add 1 to each word count to avoid zero probabilities, Ensuring that unseen words do not eliminate a class during Prediction.

$\rightarrow$ used mainly in Multinomial & Naïv Bernoulli Naive Bayes

$\rightarrow$ helps especially with Small datasets

$\rightarrow$ Too much smoothing can reduce model sharpness.

Bias-Variance Trade Off in Laplace Smoothing.

$$P(\text{word} | Y) = \frac{\text{count}(\text{word}) + \alpha}{\text{Total words} + \alpha V}$$

- $\alpha \rightarrow$ Smoothing parameter
- $V =$ Vocabulary size

→ Lets understand what α is doing hyperparameter

i) When $\alpha = \text{small}$ or $\alpha = 0$: [without smoothing]

$X \Rightarrow f_1, f_2, f_3, \dots, f_n \mid Y \Rightarrow 500 \text{ samples}$

$$P(X | Y) \Rightarrow P(Y) P(f_1 | Y) P(f_2 | Y) P(f_3 | Y) \dots$$

$$\alpha = 0 \Rightarrow \frac{1}{500} \cdot \frac{0}{500} \cdot \frac{2}{500} \Rightarrow \underline{0}$$

$$\alpha = 0.001 \Rightarrow \frac{1}{500} \cdot \frac{0.001}{500} \cdot \frac{2}{500} \Rightarrow 0.00063$$

⇒ Small changes leads to high variance.

→ If a word never appears in a class → prob become 0

→ One Zero prob makes the entire posterior prob 0

→ model becomes extremely confident about training data

→ Highly sensitive to small data

→ Leads to overfitting

→ High Variance, low Bias, poor generalization on unseen data

ii) When $\alpha > 0$ [without smoothing]

$\alpha =$ very high

$$\alpha = 1000 \Rightarrow \frac{1 + 1000}{500 + 2000} \Rightarrow \frac{1001}{2500} = \frac{10}{25} \Rightarrow \frac{2}{5}$$

$$\alpha = 10,000 \Rightarrow \frac{1 + 10000}{500 + 20000} \Rightarrow \frac{10001}{20000} \Rightarrow \frac{1}{2}$$

$$\alpha = 100,000 \Rightarrow \frac{1 + 100000}{500 + 200000} \Rightarrow \frac{100001}{200000} \Rightarrow \frac{1}{2}$$

→ Every word gets Equal probability

→ model stops learning meaningful patterns, Becomes overly 'safe' and conservative

→ Bias becomes very high (strong incorrect assumption)

→ Variance become very low

→ Leads to underfitting

$$P(X|Y) \Rightarrow \frac{P(Y) P(f_1|Y) P(f_2|Y) \dots P(f_n|Y)}{P(N) P(f_1|N) P(f_2|N) \dots P(f_n|N)} \Rightarrow \frac{1/2}{1/2} \Rightarrow \text{Same}$$

Posterior probability $\Rightarrow$ for class becomes very similar

Model mostly depends on prior $P(Y), P(N)$

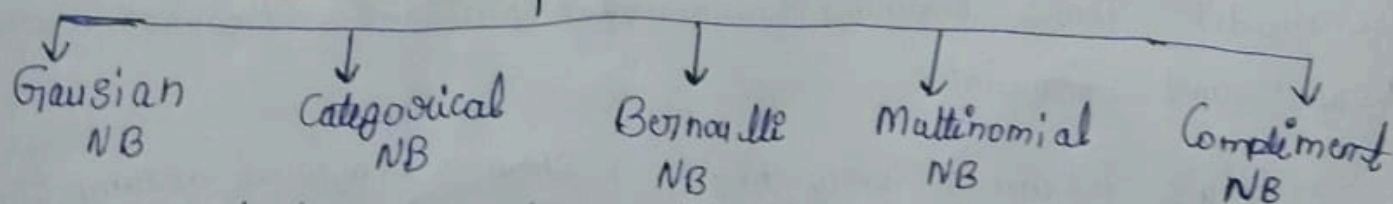
Feature contribute very little.

Laplace is applied to all words in a vocabulary for that class not only to the missing ones

Need of Laplace Smoothing:-

- i) Prevents Zero Probability Problem
- ii) Handles unseen words in Test data:- In test data, may contain new word not seen in training, Smoothing ensures they still get a small probability
- iii) Improves Generalization:- without smoothing, model becomes too confident about training data.
 $\rightarrow$ Smoothing makes model more cautious and realistic
- iv) Reduces Variance:- Probability estimates from small datasets can fluctuate heavily. Smoothing stabilizes these estimates
- v) Useful for small datasets:- not always for big dataset (model already stable)
 when training data is limited, zero counts are common, smoothing is important in such cases
- vi) acts like L2- Regularization $\Rightarrow$ Reduces Overconfidence

Types of Naive Bayes



(we use smoothing even for large dataset, but its effect becomes smaller as dataset size increases)

#1] Gaussian Naive Bayes:-

(High bias, low variance)

Assumption:- It assumes that each feature follows a Gaussian (Normal) distribution for each class.

$$X_i | Y \sim N(\mu, \sigma^2)$$

during Training

- Compute Prior $P(Y)$
- for each feature & each class, calculates mean, variance.
- store these value

during Testing

- takes new numerical data / value
 - Plug into the Gaussian Probability density function:
- $$P(X_i | Y) = \frac{1}{\sqrt{2\pi}\sigma^2} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$$
- multiply all probability ^{of} features with $P(Y)$
 - choose class with highest posterior (Placed) _{cgpa}

Ex:- data $\Rightarrow$ all features are numerical (height, weight, salary)

cgpa	iq	Placement
1. 1000 students		
500 Y		500 N

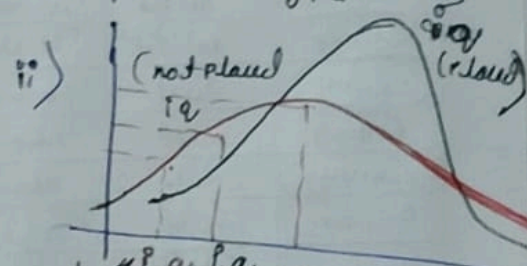
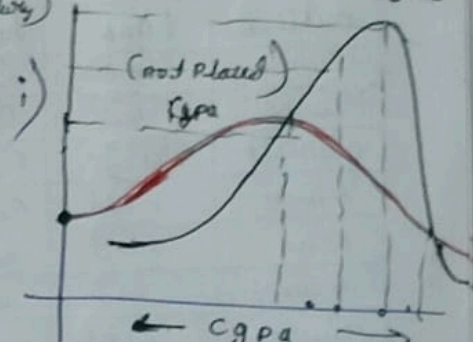
new data $\Rightarrow \{8.1, 81\} \rightarrow Y / N$

i)

cgpa	iq	Placement
μ_8	μ_{81}	Y

ii)

cgpa	iq	Placement
μ_8	μ_{81}	N



$$P(Y | \text{cgpa} = 8.1, \text{iq} = 81) \Rightarrow \overset{1/2}{P(Y)} P(\text{cgpa} = 8.1 | Y) P(\text{iq} = 81 | Y)$$

$$P(N | \text{cgpa} = 8.1, \text{iq} = 81) \Rightarrow \overset{1/2}{P(N)} P(\text{cgpa} = 8.1 | N) P(\text{iq} = 81 | N)$$

* Why don't we use Laplace Smoothing:- (used for discrete count based)
 Laplace Smoothing used to fix zero probabilities in count-based models. Gaussian Naive Bayes does not use counts $\rightarrow$ It uses a Continuous Probability density function $\rightarrow$ So zero probability due to unseen values does not occur. Even if a value never seen before, the Gaussian formula still gives a small prob.

2] Categorical Naive Bayes:-

Categorical Naive Bayes is used for discrete features where each feature takes one category value, and it estimates class-conditional probability using frequency counts with smoothing.

during training:-

i) Compute $P(Y)$

ii) Compute:-
$$P(X_i | Y) = \frac{P(X_i) \cdot \text{Count}(\text{value} | Y) + \alpha}{\text{Total samples in class } Y + \alpha K}$$

→ $K \rightarrow$ no. of categories for that feature

→ $\alpha \Rightarrow$ Smoothing parameter

→ during training we compute prior and likelihood probabilities with smoothing. During testing, we use those stored probabilities and calculate posterior prob for each class. The class with highest prob is predicted.

The class with highest prob is predicted.

Testing

new data

$X_a = \{ \text{Sunny, Hot, High, False} \}$

$$P(Y) \Rightarrow 9/14$$

$$P(N) = 5/14$$

outlook	temp	humidity	windy	Play Tennis
Sunny	Hot	High	False	No
Sunny	Hot	High	True	No
Overcast	Hot	High	F	Yes
Rainy	mild	High	F	Yes
Rainy	cool	Normal	F	Yes
Rainy	cool	Normal	T	No
Overcast	Normal	Normal	T	Yes
Sunny	mild	High	F	No
Sunny	cool	Normal	F	Yes
Rainy	mild	Normal	F	Yes
Sunny	mild	Normal	T	Yes
Overcast	mild	High	T	Yes
Overcast	Hot	Normal	F	Yes
Rainy	mild	High	T	No

for class Yes:-

$$P(\text{temp} = \text{hot} | Y) = \frac{2+1}{9+3} = \frac{3}{12} = \frac{1}{4}$$

$$P(\text{humidity} = \text{high} | Y) = \frac{3+1}{9+2} = \frac{4}{11}$$

$$P(\text{wind} = \text{False} | Y) = \frac{5+1}{9+2} = \frac{6}{11}$$

for class No:-

$$P(\text{out} = \text{Sunny} | N) = \frac{3+1}{5+3} = \frac{4}{8} = \frac{1}{2}$$

$$P(\text{temp} = \text{hot} | N) = \frac{2+1}{5+3} = \frac{3}{8}$$

$$i) P(Y | X_a) = P(Y) \cdot P(\text{outlook} = \text{Sunny} | Y)$$

$$P(\text{temp} = \text{hot} | Y) \cdot P(\text{humidity} = \text{High} | Y)$$

$$P(\text{windy} = \text{False} | Y)$$

$$ii) P(N | X_a) = P(N) \cdot P(\text{outlook} = \text{Sunny} | N)$$

$$P(\text{temp} = \text{hot} | N) \cdot P(\text{humidity} = \text{High} | N)$$

$$P(\text{windy} = \text{False} | N) \Rightarrow \frac{5}{2}$$

$$P(\text{hum} = \text{High} | N) = \frac{4+1}{5+2}$$

$$P(\text{windy} = \text{False} | N) = \frac{2+1}{5+2} = \frac{3}{7}$$

$$P(Y|X_4) = (9/14) (1/4) (1/4) (4/11) (6/11) = 0.00796$$

$$P(N|X_4) = (5/14) (1/2) (3/8) (5/7) (3/7) \Rightarrow \boxed{0.0204}$$

$\Rightarrow$ class No is predicted

#3] Multinomial Naive Bayes:- (discrete counts)

used mainly for textual data (Bag of words (Bow) | $\frac{\text{TF-counts}}{\text{fraction}}$)

where it is used

$\rightarrow$ spam detection $\rightarrow$ sentiment analysis $\rightarrow$ News classification

$\rightarrow$ document classification

$\rightarrow$ Multinomial NB works on

• word counts

• Frequency of words in documents

Bow $\rightarrow$ binary
 $\rightarrow$ non-binary (counts)

movie review	Sentiment
---	!
---	!

great	fair	epic
2	0	0
0	3	2
1	1	1

great	fair	epic
1	0	0
0	1	1
1	1	1

Ex:- Classify Email as Spam or not Spam

Email text	class
"win money now"	Spam
"win cash prize"	Spam
"meeting schedule now"	Not Spam
"project meeting schedule"	Not Spam

vocabulary = { win, me
now, cash, prize, meeting
schedule, project }

Spam class

$\rightarrow$ win money now } word counts
 $\rightarrow$ win cash prize }

word	count
win	2
money	2
now	1
cash	1
prize	1
meeting	0
Schedule	0
project	0

$n=5$

Total words in
Spam = 5 //

Not Spam class

word	count
meeting	2
Schedule	2
now	1
project	1
meeting win	0
meeting money	0
cash	0
prize	0

Total
words
not Sp

Apply Laplace Smoothing: $(\alpha=1)$

$$P(\text{word} | \text{class}) = \frac{\text{count} + 1}{\text{Total words in each class} + \alpha n}$$

($n \Rightarrow$ total vocabulary size = 8)

Spam: $P(\text{Spam}) \Rightarrow 2/4 = 0.5$; $P(\text{not Spam}) \Rightarrow 0.5$

find conditional Probabilities for Each class

Spam: $P(\text{win} | \text{Spam}) = \frac{2+1}{6+8} = 3/14$

$P(\text{meeting} | \text{Spam}) = \frac{0+1}{6+8} = 1/14$

$P(\text{money} | \text{Spam}) = \frac{2+1}{6+8} = 3/14$

$P(\text{cash} | \text{Spam}) = \frac{2}{6+8} = 2/14$

Not Spam

$P(\text{win} | \text{not Spam}) = \frac{0+1}{6+8} = 1/14$

$P(\text{money} | \text{not Spam}) = \frac{1}{6+8} = 1/14$

$P(\text{cash} | \text{not Spam}) = 1/14$

$P(\text{meeting} | \text{not Spam}) = 3/14$

like this calculate all possible probab

→ testing: "win huge cash prize now!"

(Since not calculated likelihood during training)
Ignore

Spam

$P(\text{win} | \text{Spam})' = 3/14$

$P(\text{huge} | S)' = 1/14$, $P(\text{now} | S)' = 2/14$

$P(\text{cash} | S)' = 2/14 \Rightarrow 2/7$

$P(\text{prize} | S)' \Rightarrow 2/14 \Rightarrow 2/7$

Not Spam

$P(\text{win} | \text{NS})' = 1/14$ ignore

$P(\text{huge} | \text{NS})' = 1/14$

$P(\text{cash} | \text{NS})' = 1/14$

$P(\text{prize} | \text{NS})' = 1/14$

$P(\text{now} | \text{NS})' = 2/14 \Rightarrow 1/7$

Posterior for Spam $\Rightarrow 0.5 \times (3/14) \times (1/14) \times (2/7) \times (2/7)$

$= 0.000156$ } apply log

Posterior for not Spam (NS) $\Rightarrow 0.5 \times (1/14) \times (1/14) \times (1/14) \times (1/7)$

$= 0.0000185$

finally Spam is predicted

#2 Binomial Distribution:-

- multiple Bernoulli independent trials
- fixed no. of trials (n)
- Same probability P
- Count no. of success

Ex:-

- Toss coin 5 times
- How many heads?

Probability $\Rightarrow$

$$P(X=k) = {}^nC_k P^k (1-P)^{n-k}$$

$n \Rightarrow$ no. of trials
 $k \Rightarrow$ no. of success

- Bernoulli $\Rightarrow$ 1 coin toss
- Binomial $\Rightarrow$ many coin toss (but only 2 outcomes)

#3 Multinomial Distribution:-

- Instead of 2 outcomes, we have multiple categories

Ex:- Rolling a die, word selection from vocabulary, Email words belonging to diff categories.

Probability:- if n trials, $k_1, k_2, \dots, k_n$ counts
 $p_1, p_2, \dots$ probabilities

$$P(X=k) = P_k$$

$$P = \frac{n!}{k_1! k_2! \dots k_n!} p_1^{k_1} p_2^{k_2} p_3^{k_3}$$

- Binomial $\Rightarrow$ 2 categories / outcomes
- multinomial $\Rightarrow$ 3 or more categories

#4 Bernoulli Naive Bayes:-

- Bernoulli Naive Bayes implements Naive Bayes training and classification algorithms for data that is distributed according to multivariate Bernoulli distributions.
- There may be multiple features but each one is assumed to be binary-valued (Bernoulli, Boolean) variable.

f_1	f_2	f_3	...	f_n
0	0	1		0
1	1	1		0
0	1	1		1
0	0	0		1
0	0	0		0

} $\Rightarrow$ Bag of words $\Rightarrow$ Binary

present / no

Why do we need Bernoulli?

- Because many real world situations have:
- only two possible outcomes
 - single trial

Ex:- Coin toss → Head / Tail

Email → spam / Not spam

Student → Pass / Fail, Feature present → Yes / No

→ whenever outcome is binary ⇒ Bernoulli is used

Bernoulli distribution:-

Probability of Success = P , then

$$P(X=1) \Rightarrow P$$

$$P(X=0) = 1-P$$

Laplace addition
for Bernoulli

$$P(w|c) = \frac{\text{count} + 1}{\text{Total words} + 1}$$

in each class

$$\text{Formula: } P(X=x) = P^x (1-P)^{1-x} \quad x \in \{0,1\}$$

if $x=1$:- $P^1 (1-P)^0 = P //$

if $x=0$:- $P^0 (1-P)^1 = 1-P //$

} Handles both cases

Ex:- Training data

Email	Class
win money"	Spam
free offer"	Spam
meeting today"	Not Spam
project meeting"	Not Spam

Spam

word	Yes/1
win	1
money	1
free	1
offer	1
meeting	0
project	0
today	0

Not Spam

word	Not/0
win	0
money	0
free	0
offer	0
meeting	1
project	1
today	1

Ex:- win prize money prize

	win	money	free	offer	meeting	to day	Project	Class 0/1
$d_1 \Rightarrow$	1	0	0	0	0	0	0	Spam
$d_2 \Rightarrow$	0	0	1	1	0	0	0	Spam
$d_3 \Rightarrow$	0	0	0	0	0	1	0	Not Spam
$d_4 \Rightarrow$	0	0	0	0	1	0	1	Not Spam

$$P(\text{win}=1, \text{money}=1, \text{free}=0, \text{offer}=0, \text{meeting}=0, \text{today}=0, \text{project}=0 | \text{spam})$$

$$\Rightarrow P(\text{spam})^{1/2} P(\text{win}=1 | \text{spam}) P(\text{money}=1 | \text{spam}) P(\text{free}=0 | \text{spam}) P(\text{offer}=0 | \text{spam})$$

$$P(\text{meeting}=0 | \text{spam}) P(\text{today}=0 | \text{spam}) P(\text{project}=0 | \text{spam})$$

$$\Rightarrow P(\text{win}=1 | \text{spam}) = 1/2$$

$$P(\text{offer}=0 | \text{spam}) = 1/2$$

$$\Rightarrow P(\text{money}=1 | \text{spam}) = 1/2$$

$$P(\text{meeting}=0 | \text{spam}) = 1$$

$$\Rightarrow P(\text{free}=0 | \text{spam}) = 1/2$$

$\Rightarrow$ like this do for all conditional probability

Why Bernoulli considers absence probability?

Because of how the Bernoulli distribution works.

In Bernoulli distribution:-

Each feature has only two possible outcomes:

$$x_1 = 1 \quad (\text{word present})$$

$$x_2 = 0 \quad (\text{word absent})$$

Since 0 & 1 are valid outcomes, the model must compute

$$P(x_1 = 1 | y) \quad [\text{present}]$$

$$P(x_2 = 0 | y) \Rightarrow 1 - P(x_1 = 1 | y) \quad [\text{absent}]$$

$\rightarrow$ So when computing likelihood: / Conditional probab

$$P(X|Y) = \prod P(x_i | Y)$$

$\rightarrow$ Even if the word is not in the document ($x_i = 0$)

its probability contribute as:

$$1 - P(\text{word}=1 | Y)$$

$\therefore$ Therefore absence directly affects the probability

Ex:- spam detection [absence is informative]

- If Spam emails usually contain "free"
- and this (new) email does not contain "free"
- that is useful information | So absence must be modeled.

Why Multinomial does not consider absence?

Because of how the multinomial distribution works.

→ Multinomial model counts of words drawn from a class distribution.

→ The formula is: $P(x|y) = \prod P(\text{word} | y)^{\text{count}}$ how many types is applied

Now observe → If count = 0

$$P(\text{word} | y)^0 \rightarrow 1$$

→ This term becomes 1 & disappears

• So, words not present in the document, do not change the probability

• They are ignored automatically

→ The model only cares about how many times word appear not about missing words

Power = 0 $(P(x|y) = \prod P(\text{word} | y)^0) \Rightarrow$ a word in vocabulary confusion but does not appear in the test document

Situation

→ word seen in training (vocabulary) but not in this class

→ word never seen in entire training data (vocabulary)

What to do

→ apply Laplace smoothing

Ignore it (power = count = 0) which means:

→ In real ML systems (like sklearn), we don't truly ignore words manually. Instead

vocab is fixed during training, during testing, words in the vocab are automatically dropped.

advanced systems add special token UNK token or use techniques like feature hashing

Bernoulli detailed Example:-

Email	Text	Label
D1	win money now	Spam
D2	win prize	Spam
D3	hello friend	not spam
D4	hello money	not spam

$V = \{ \text{win, money, prize, hello, friend, now} \}$

Vocabulary size = 6

word in binary $\Rightarrow$

	Email	win	money	now	prize	hello	friend	Label
1	d1	1	1	1	0	0	0	Spam
2	d2	1	0	0	1	0	0	Spam
3	d3	0	0	0	0	1	1	not spam
2	d4	0	1	0	0	1	0	not spam
(test) d5				0	0	0	0	

no. of Spam doc = 2
no. of not spam doc = 2

test data = "win money"

Apply Laplace Smoothing ($\alpha=1$)

$$P(\text{word} = i | y) = \frac{\text{Count} + \alpha}{(N) + 2}$$

why +2:- Because Bernoulli $\Rightarrow$ binary } possible values = $\{0, 1\}$

for Spam :- $N=2$

$$P(\text{win} = 1 | \text{spam}) = \left(\frac{2 + 1/2}{2 + 2} \right) = 3/4$$

$$P(\text{money} = 1 | \text{spam}) = \left(\frac{1 + 1/2}{2 + 2} \right) = 1/4$$

$$P(\text{now} = 1 | \text{spam}) = \left(\frac{1 + 1/2}{2 + 2} \right) = 1/4$$

$$P(\text{prize} = 1 | \text{spam}) = \left(\frac{1 + 1/2}{2 + 2} \right) = 1/4$$

$$P(\text{hello} = 1 | \text{spam}) = \left(\frac{0 + 1/2}{2 + 2} \right) = 1/8$$

$$P(\text{friend} = 1 | \text{spam}) = 1/8$$

for not spam $\Rightarrow N=2$

$$P(\text{win} = 1 | \text{not spam}) = \left(\frac{0 + 1/2}{2 + 2} \right) = 1/8$$

$$P(\text{money} = 1 | \text{ns}) = \left(\frac{1 + 1/2}{2 + 2} \right) = 3/8$$

$$P(\text{now} = 1 | \text{ns}) = \left(\frac{0 + 1/2}{2 + 2} \right) = 1/8$$

$$P(\text{prize} = 1 | \text{ns}) = \left(\frac{0 + 1/2}{2 + 2} \right) = 1/8$$

$$P(\text{hello} = 1 | \text{ns}) = \left(\frac{2 + 1/2}{2 + 2} \right) = 5/8$$

$$P(\text{friend} = 1 | \text{ns}) = \left(\frac{1 + 1/2}{2 + 2} \right) = 3/8$$

for absent $\Rightarrow$

$$1 - P(i)$$

test data: 'win money'

win	money	now	prize	hello	friend	if 0 $\Rightarrow$ 1-p
1	1	0	0	0	0	

Posterior $\Rightarrow P(y) \Rightarrow \left\{ \begin{array}{l} P(\text{spam}) = 0.5 \\ P(\text{ns}) = 0.5 \end{array} \right\}$

Posterior $P(y|x) \propto P(y) \times \prod P(x_i|y)$

Spam

$P(\text{win} = 1 | \text{spam}) = 3/4$ } from train data

$P(\text{money} = 1 | \text{spam}) = 2/4$

$P(\text{now} = 0 | \text{spam}) \Rightarrow 1 - P(\text{now} = 1 | \text{spam})$
 $\Rightarrow 1 - 2/4 = 2/4$

$P(\text{prize} = 0 | \text{spam}) \Rightarrow 2/4$

$P(\text{hello} = 0 | \text{spam}) \Rightarrow 3/4$

$P(\text{friend} = 0 | \text{spam}) \Rightarrow 3/4$

$0.5 \times 3/4 \times 2/4 \times 2/4 \times 2/4 \times 3/4 \times 3/4$

not spam

$P(\text{win} = 1 | \text{ns}) = P(\text{ns})$
 $= 0.5/1 = 1/4$

$P(\text{money} = 1 | \text{ns}) = 2/4$

$P(\text{now} = 0 | \text{ns}) = 1 - 1/4 = 3/4$

$P(\text{prize} = 0 | \text{ns}) = 3/4$

$P(\text{hello} = 0 | \text{ns}) = 1 - 3/4 = 1/4$

$P(\text{friend} = 0 | \text{ns}) = 1 - 2/4 = 2/4$

$0.5 \times 1/4 \times 2/4 \times 3/4 \times 1/4 \times 2/4$

Spam Probab > Not spam Probab

Apply Laplace correction for small dataset

$\rightarrow$ why this happens in more in small dataset

Because, Vocabulary size is large

documents are few

many words will have zero frequency in some class

this is called data sparsity problem.

Sparsity means most values in your dataset are zero

10,000 unique words (vocabulary size = 10,000)

one doc contains only 20 words

[1 0 0 0 1 0 0 1 0 1 1 1 0 0 0 0]

almost all
entries = 0